Architecture 1001: x86-64 Assembly CallASubroutine1.c

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

CallASubroutine1.c

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

CallASubroutine1.c

```c
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

```
func:
0000000140001000  mov          eax,0BEEFh
0000000140001005  ret
main:
0000000140001010  sub          rsp,28h
0000000140001014  call         func (0140001000h)
0000000140001019  mov          eax,0F00Dh
000000014000101E  add          rsp,28h
0000000140001022  ret
```

CallASubroutine1.c

```
int func(){
    return 0xbeef;
}
int main(){
    func();
    return 0xf00d;
}
```

```
func:
0000000140001000  mov          eax,0BEEFh
0000000140001005  ret
main:
0000000140001010  sub          rsp,28h
0000000140001014  call         func (0140001000h)
0000000140001019  mov          eax,0F00Dh
000000014000101E  add          rsp,28h
0000000140001022  ret
```





CALL - Call Procedure

- CALL's job is to transfer control to a different function; in a way that control can later be resumed where it left off
- First it *pushes* the address of the next instruction onto the stack
  - For use by RET for when the procedure is done
- Then it changes RIP to the address given in the instruction

Book page 108



CALL - Call Procedure

- Destination address for the target function can be specified in multiple ways
  - Absolute address
  - Relative address (relative to the end of the instruction, or some other register)

Book page 108



RET - Return from Procedure

- Two forms
  - Pop the top of the stack into RIP (remember, *pop* implicitly increments stack pointer, RSP)
    - In this form, the instruction is just written as “ret”
  - Pop the top of the stack into RIP and also add a constant number of bytes to RSP
    - In this form, the instruction is written as “ret 0x8”, or “ret 0x20”, etc

Book page 109

How to read two-operand instructions: Intel vs. AT&T Syntax

- Intel: Destination <- Source(s)
  - Windows. Think algebra or C: y = 2x + 1;
  - mov rbp, rsp
  - add rsp, 0x14 ; (rsp = rsp + 0x14)
- AT&T: Source(s) -> Destination
  - *nix/GNU. Think elementary school: 1 + 2 = 3
  - mov %rsp, %rbp
  - add $0x14,%rsp
  - So registers get a % prefix and immediates get a $
- My classes will prefer Intel syntax
- It's important to know both, so you can read documents in either format



MOV - Move

- Can move:
  - register to register
  - memory to register, register to memory
  - immediate to register, immediate to memory
- Never memory to memory!
- Memory addresses are given in “r/mX” form

“r/mX” Addressing Example

immediate to memory	immediate to register
`mov [rbx], imm32`	`mov rbx, imm64`
`mov [rbx+rcx*X], imm32`	**register to register**
`mov [rbx+rcx*X+Y], imm32`	`mov rbx, rax`
register to memory	**memory to register**
`mov [rbx], rax`	`mov rax, [rbx]`
`mov [rbx+rcx*X], rax`	`mov rax, [rbx+rcx*X]`
`mov [rbx+rcx*X+Y], rax`	`mov rax, [rbx+rcx*X+Y]`

Note: there’s nothing special about the particular registers I chose above.
Any could be substituted with any other general purpose registers



ADD and SUB

- Adds or Subtracts, just as expected
- Destination operand can be r/mX or register
- Source operand can be r/mX or register or immediate
- No source ***and*** destination as r/mXs, because that could allow for memory to memory transfer, which isn't allowed on x86

- `add rsp, 8` → (rsp = rsp + 8)
- `sub rax, [rbx*2]` → (rax = rax - memorypointedtoby(rbx*2))

Book page 59 = Add, page 62 = Sub

CallASubroutine1.c 1:

RIP = 00000001`40001010, but no *instruction yet executed*

Key:

- ⍓ **executed instruction,**
- ♍ **modified value**
- ⌘ **start value**

```
func:
0000000140001000  mov   eax,0BEEFh
0000000140001005  ret
main:
0000000140001010  sub   rsp,28h
0000000140001014  call  func (0140001000h)
0000000140001019  mov   eax,0F00Dh
000000014000101E  add   rsp,28h
0000000140001022  ret
```

Registers

rax	00000000`00000001 ⌘
rsp	00000000`0014FE08 ⌘

Stack

rsp→ 00000000`0014FE08	00000001`40001349
…	…
00000000`0014FDE0	undef
00000000`0014FDD	undef
00000000`0014FD 0	undef
00000000`0014F C8	undef

Belongs to the frame *before* main() is called

CallASubroutine1.c 2:

Key:

⌧ **executed instruction,**

♍ **modified value**

⌘ **start value**

```
func:
0000000140001000   mov    eax,0BEEFh
0000000140001005   ret
main:
0000000140001010⌧  sub    rsp,28h
0000000140001014   call   func (0140001000h)
0000000140001019   mov    eax,0F00Dh
000000014000101E   add    rsp,28h
0000000140001022   ret
```

Registers

rax	00000000`00000001 ⌘
rsp	00000000`0014FDE0 ♍

Stack

00000000`0014FE08	00000001`40001349
…	…
rsp → 00000000`0014FDE0	undef
00000000`0014FDD8	undef
00000000`0014FDD0	undef
00000000`0014FDC8	undef

CallASubroutine1.c 3:

Key:

executed instruction,

modified value

start value

```
func:
0000000140001000   mov    eax,0BEEFh
0000000140001005   ret
main:
0000000140001010   sub    rsp,28h
0000000140001014   call   func(0140001000h)
0000000140001019   mov    eax,0F00Dh
000000014000101E   add    rsp,28h
0000000140001022   ret
```

Registers

rax	00000000`00000001
rsp	00000000`0014FDD8

Stack

00000000`0014FE08	00000001`40001349
…	…
00000000`0014FDE0	undef
rsp → 00000000`0014FDD8	00000001`40001019
00000000`0014FDD0	undef
00000000`0014FDC8	undef

CallASubroutine1.c 4:

Key:

♍ **executed instruction,**

♍ **modified value**

⌘ **start value**

```
func:
0000000140001000 ♍ mov    eax,0BEEFh
0000000140001005   ret
main:
0000000140001010   sub    rsp,28h
0000000140001014   call   func (0140001000h)
0000000140001019   mov    eax,0F00Dh
000000014000101E   add    rsp,28h
0000000140001022   ret
```

Registers

rax	00000000`0000BEEF ♍
rsp	00000000`0014FDD8

Stack

00000000`0014FE08	00000001`40001349
…	…
00000000`0014FDE0	undef
rsp → 00000000`0014FDD8	00000001`40001019
00000000`0014FDD0	undef
00000000`0014FDC8	undef

CallASubroutine1.c 4:

You can't tell, but it "zero extended" the rax reg (meaning it filled in the upper 32 bits of rax with zeros)

Registers

rax	00000000`0000BEEF ♍
rsp	00000000`0014FDD8

From section 3.4.1.1 in the Nov 2020 Intel Manual included with class materials:

When in 64-bit mode, operand size determines the number of valid bits in the destination general-purpose register:

- 64-bit operands generate a 64-bit result in the destination general-purpose register.
- 32-bit operands generate a 32-bit result, zero-extended to a 64-bit result in the destination general-purpose register.
- 8-bit and 16-bit operands generate an 8-bit or 16-bit result. The upper 56 bits or 48 bits (respectively) of the destination general-purpose register are not modified by the operation. If the result of an 8-bit or 16-bit operation is intended for 64-bit address calculation, explicitly sign-extend the register to the full 64-bits.

NOTE: This only applies to writing to *registers*, not memory!
If you write a 32 bit value to what you're imagining as a "64 bit" memory location (such as for a 64 bit local variable), still only 32 bits will be changed!

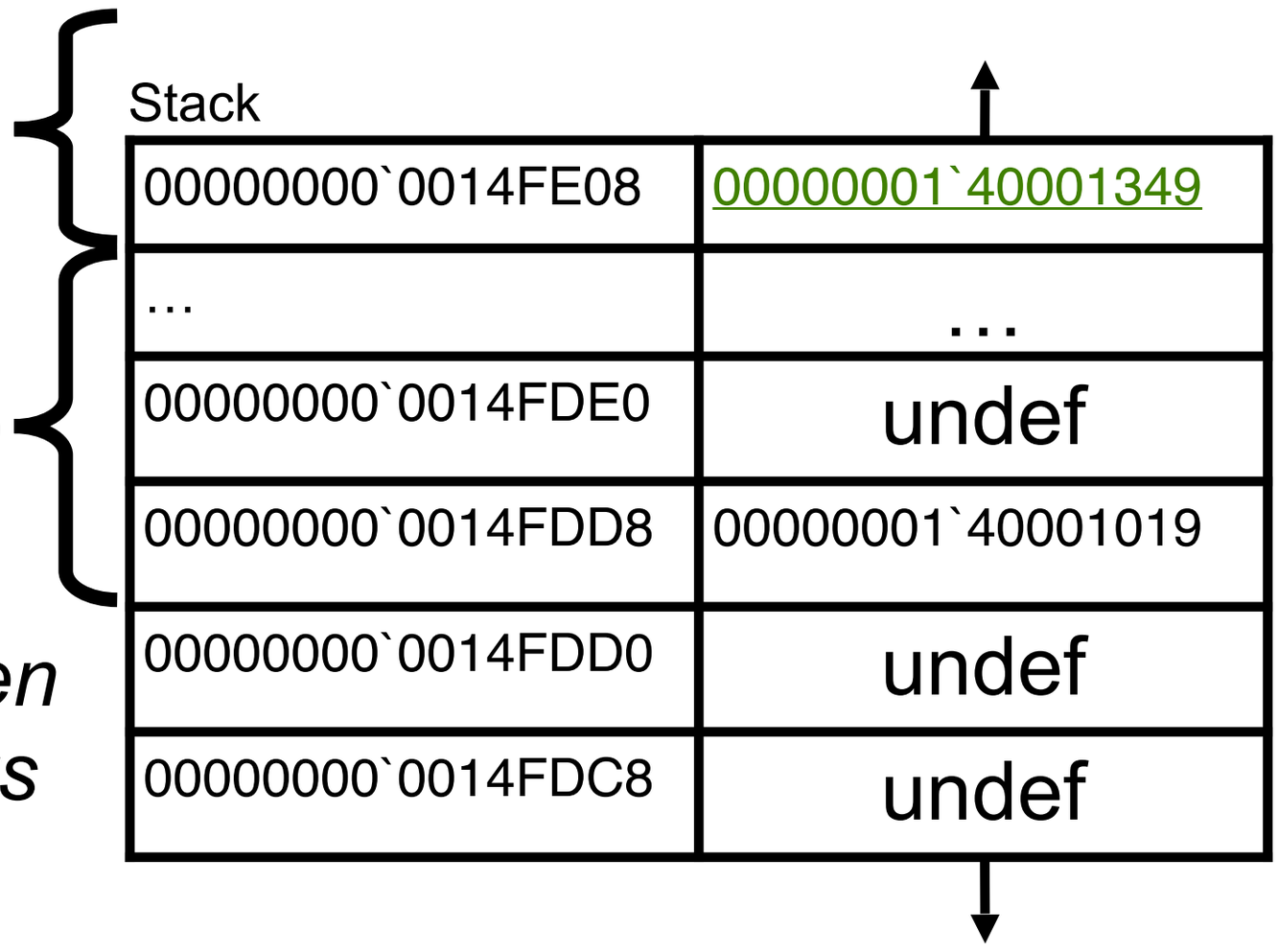
CallASubroutine1.c: STACK FRAME TIME OUT

```
func:
mov    eax,0BEEFh
ret
main:
sub    rsp,28h
call   func (0140001000h)
mov    eax,0F00Dh
add    rsp,28h
ret
```

"Function-before-main""s frame

main's frame

func never even makes/uses its stack frame



CallASubroutine1.c 5:

Key:

☒ **executed instruction,**

♍ **modified value**

⌘ **start value**

```
func:
0000000140001000  mov    eax,0BEEFh
0000000140001005☒ ret
main:
0000000140001010  sub    rsp,28h
0000000140001014  call   func (0140001000h)
0000000140001019  mov    eax,0F00Dh
000000014000101E  add    rsp,28h
0000000140001022  ret
```

Registers

rax	00000000`0000BEEF
rsp	00000000`0014FDE0 ♍

Stack

00000000`0014FE08	00000001`40001349
…	…
rsp→ 00000000`0014FDE0	undef
00000000`0014FDD8	undef ♍
00000000`0014FDD0	undef
00000000`0014FDC8	undef

CallASubroutine1.c 6:

Key:

⌧ **executed instruction,**

♍ **modified value**

⌘ **start value**

```
func:
0000000140001000   mov    eax,0BEEFh
0000000140001005   ret
main:
0000000140001010   sub    rsp,28h
0000000140001014   call   func (0140001000h)
0000000140001019 ⌧ mov    eax,0F00Dh
000000014000101E   add    rsp,28h
0000000140001022   ret
```

Registers

rax	00000000`0000F00D ♍
rsp	00000000`0014FDE0

Stack

00000000`0014FE08	00000001`40001349
…	…
rsp → 00000000`0014FDE0	undef
00000000`0014FDD8	undef
00000000`0014FDD0	undef
00000000`0014FDC8	undef

CallASubroutine1.c 7:

Key:

⍓ **executed instruction,**

♍ **modified value**

⌘ **start value**

```
func:
0000000140001000   mov    eax,0BEEFh
0000000140001005   ret
main:
0000000140001010   sub    rsp,28h
0000000140001014   call   func (0140001000h)
0000000140001019   mov    eax,0F00Dh
000000014000101E⍓  add    rsp,28h
0000000140001022   ret
```

Registers

rax	00000000`0000F00D
rsp	00000000`0014FE08 ♍

Stack

rsp→ 00000000`0014FE08	00000001`40001349
…	…
00000000`0014FDE0	undef
00000000`0014FDD8	undef
00000000`0014FDD0	undef
00000000`0014FDC8	undef

CallASubroutine1.c 8:

Key:

⍁ **executed instruction,**

♍ **modified value**

⌘ **start value**

```
func:
0000000140001000   mov    eax,0BEEFh
0000000140001005   ret
main:
0000000140001010   sub    rsp,28h
0000000140001014   call   func (0140001000h)
0000000140001019   mov    eax,0F00Dh
000000014000101E   add    rsp,28h
0000000140001022⍁  ret
```

Registers

rax	00000000`0000F00D
rsp	00000001`0014FE10 ♍

rsp→ Stack

00000000`0014FE08	undef ♍
…	…
00000000`0014FDE0	undef
00000000`0014FDD8	undef
00000000`0014FDD0	undef
00000000`0014FDC8	undef

Execution would continue at the value ret removed from the stack: *00000001`40001349*

CallASubroutine1 Notes

- func() is dead code - its return value is not used for anything, and main() always returns 0xF00D. If optimizations were turned on in the compiler, it would remove func()
- We don't yet understand why main() does "`sub rsp,28h`" and "`add rsp,28h`"...

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn’t?

Mystery Listery

- Why do the GCC/Clang HelloWorlds have balanced Push/Pop instructions but Visual Studio doesn't?
- What's up with the sub/add 0x28 in main()?

Simple Stack Diagram 2

stack bottom

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}
```

```c
int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

stack top

Simple Stack Diagram 2

stack bottom

"return address" **to return from main() to invoke_main()**

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

stack top

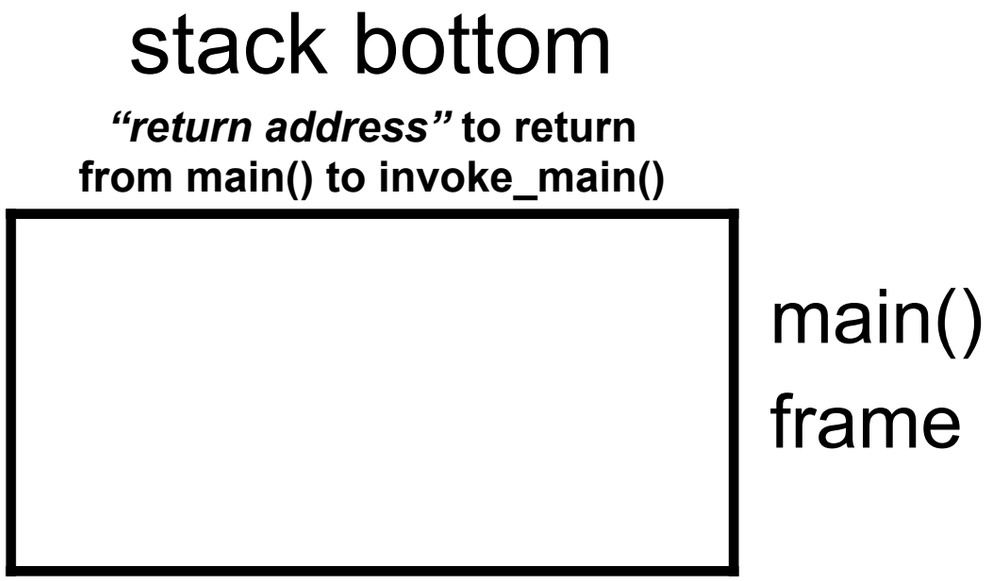
Simple Stack Diagram 2

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



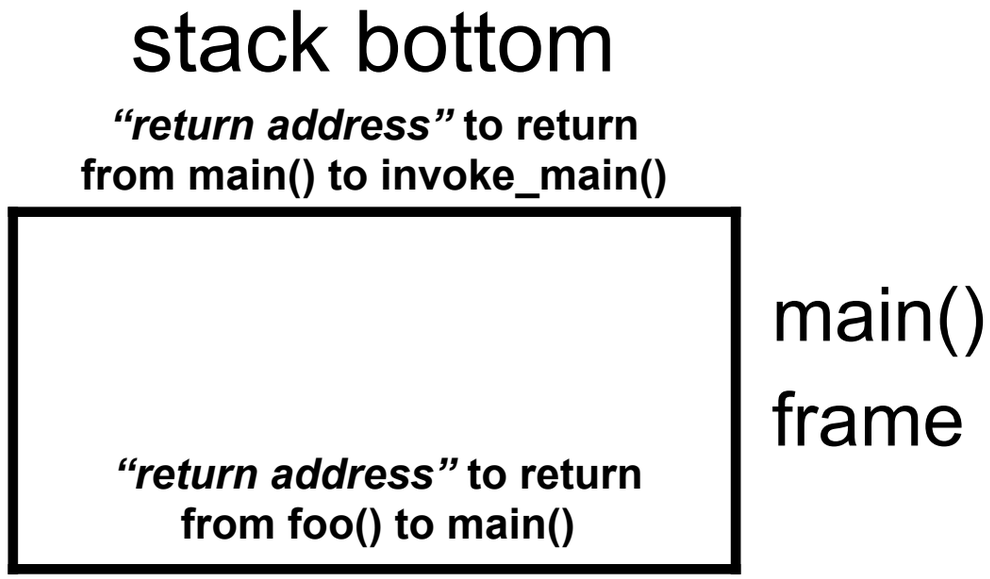
Simple Stack Diagram 2

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}
```

```c
int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



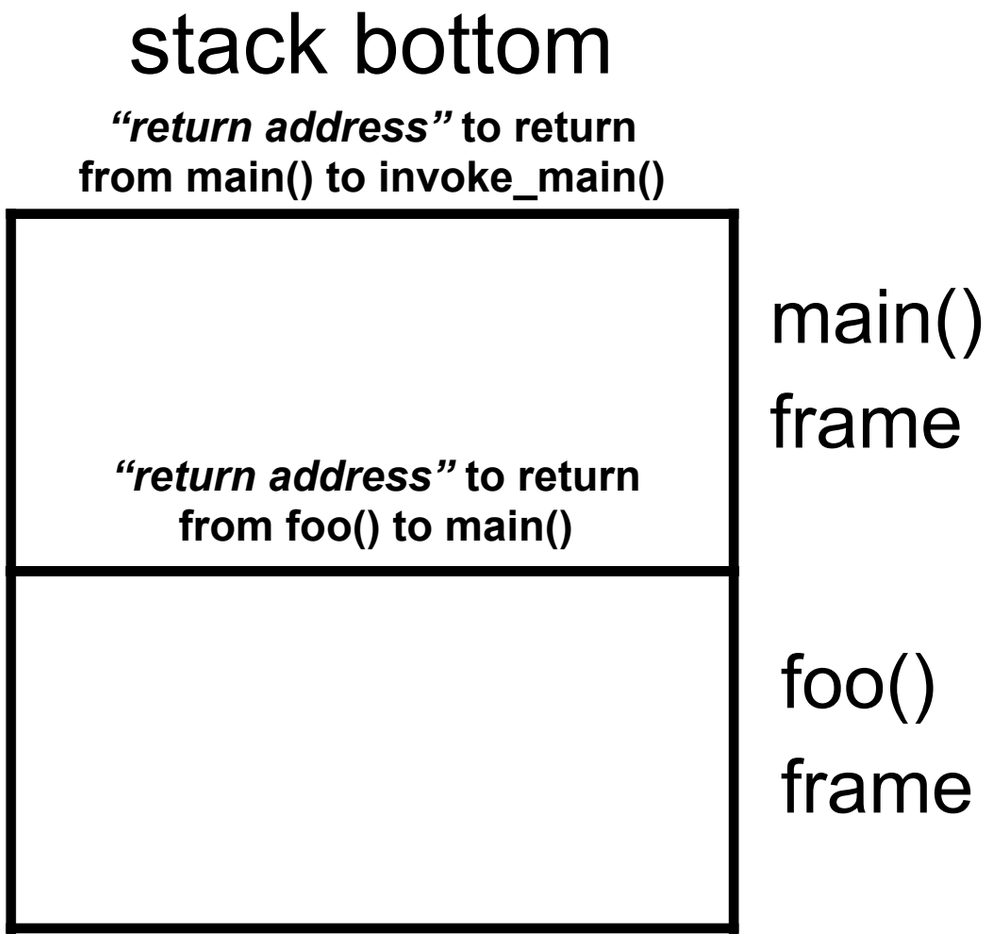
Simple Stack Diagram 2

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



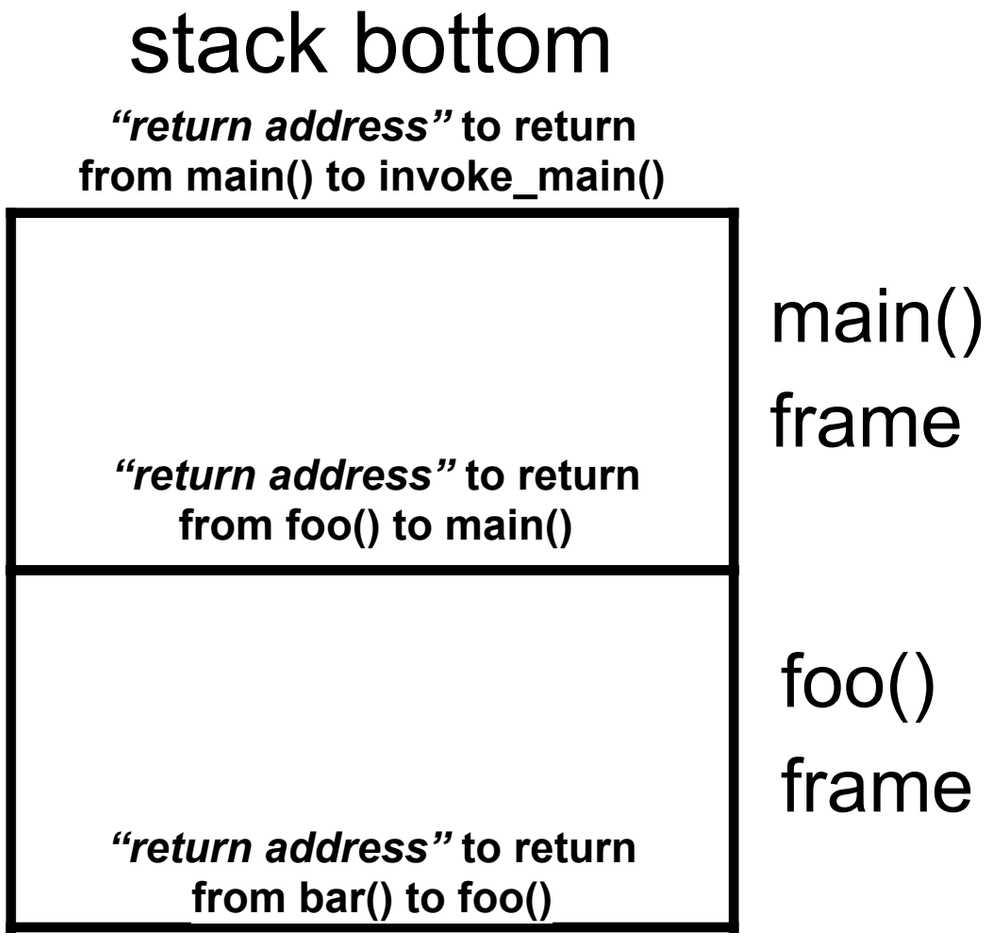
Simple Stack Diagram 2

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



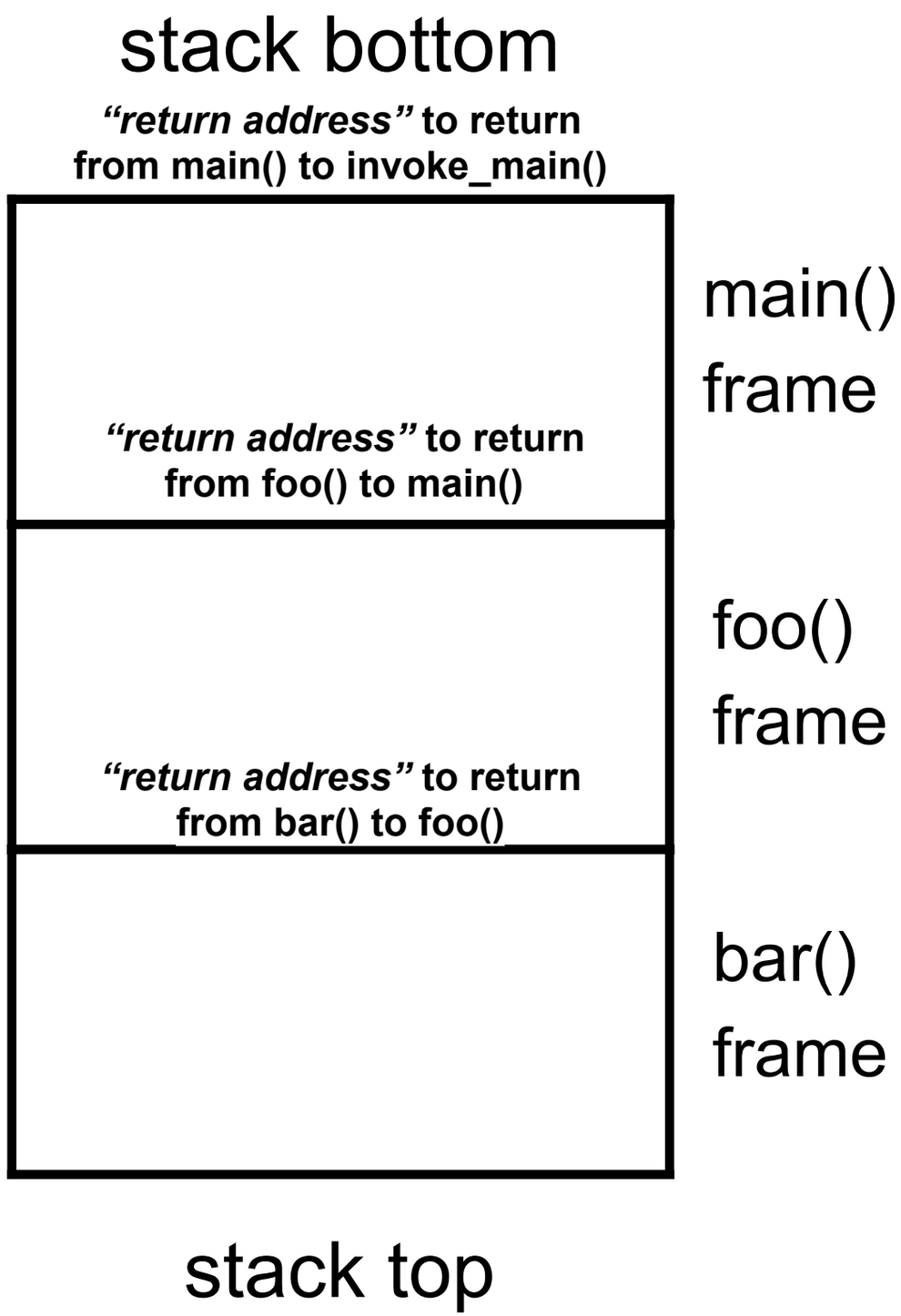
Simple Stack Diagram 2

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



Instructions we now know (8)

- NOP
- PUSH/POP
- **CALL/RET**
- **MOV**
- **ADD/SUB**

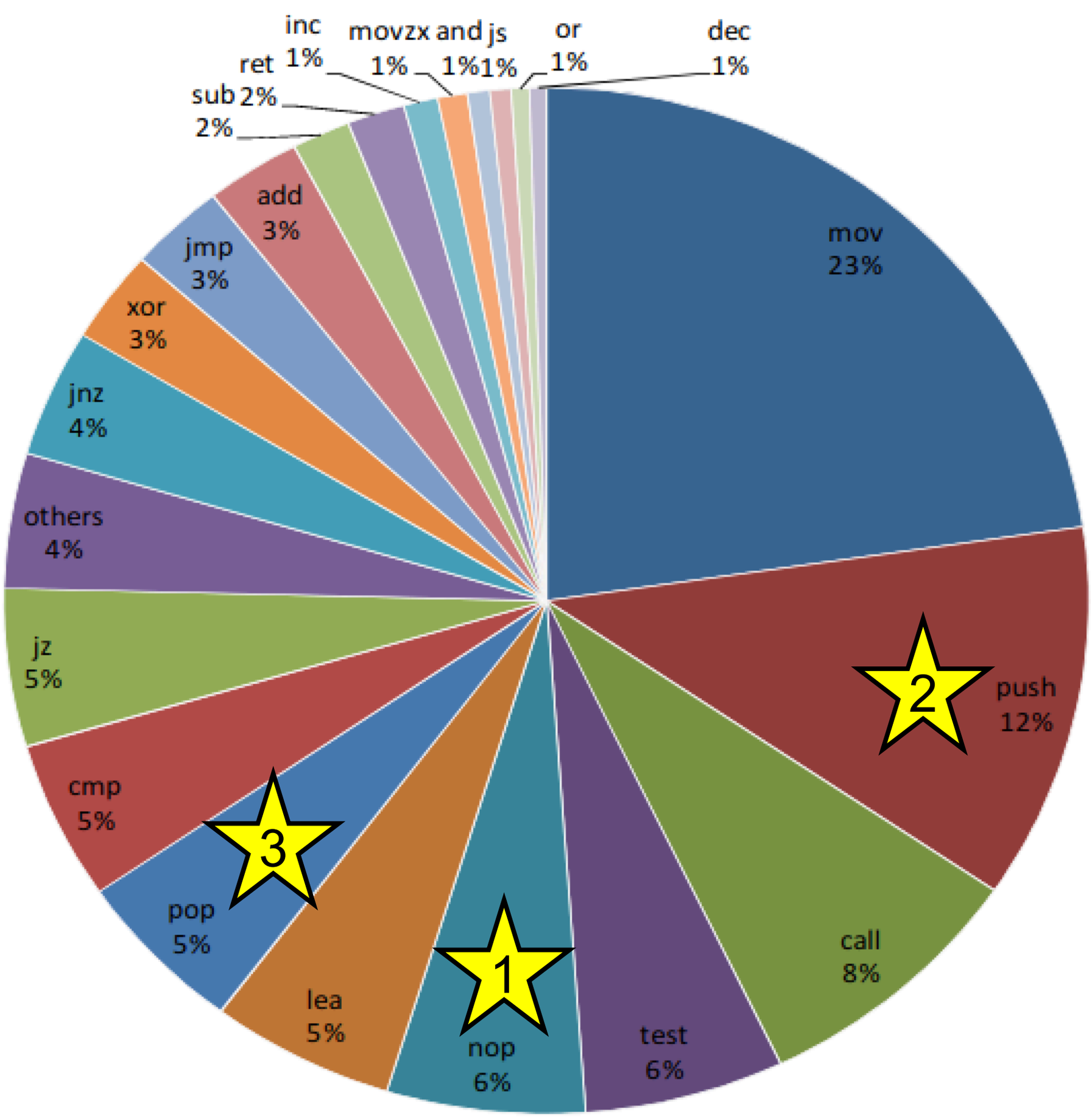


Fig. 2 Web browsers' instruction frequencies

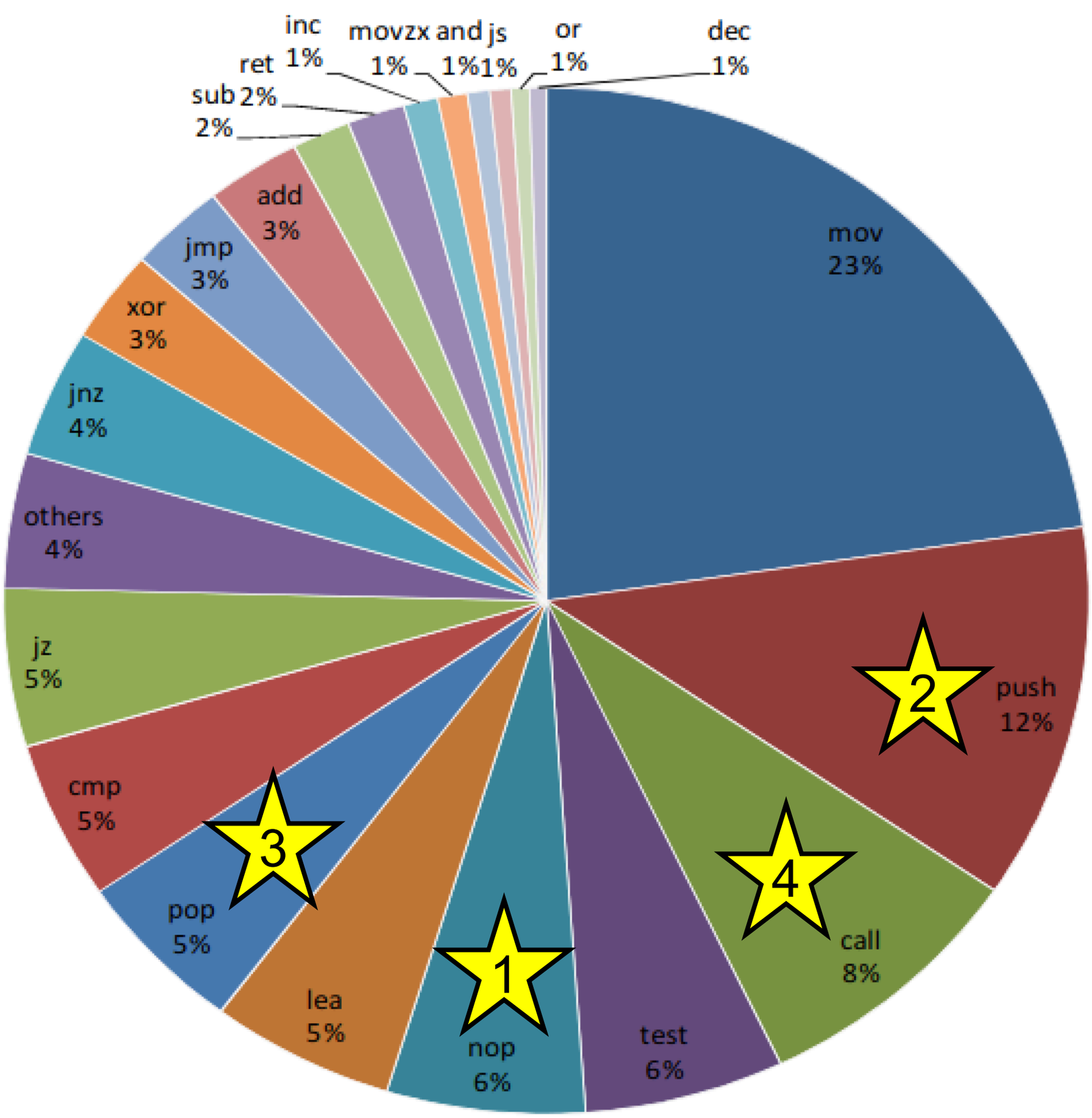


Fig. 2 Web browsers' instruction frequencies

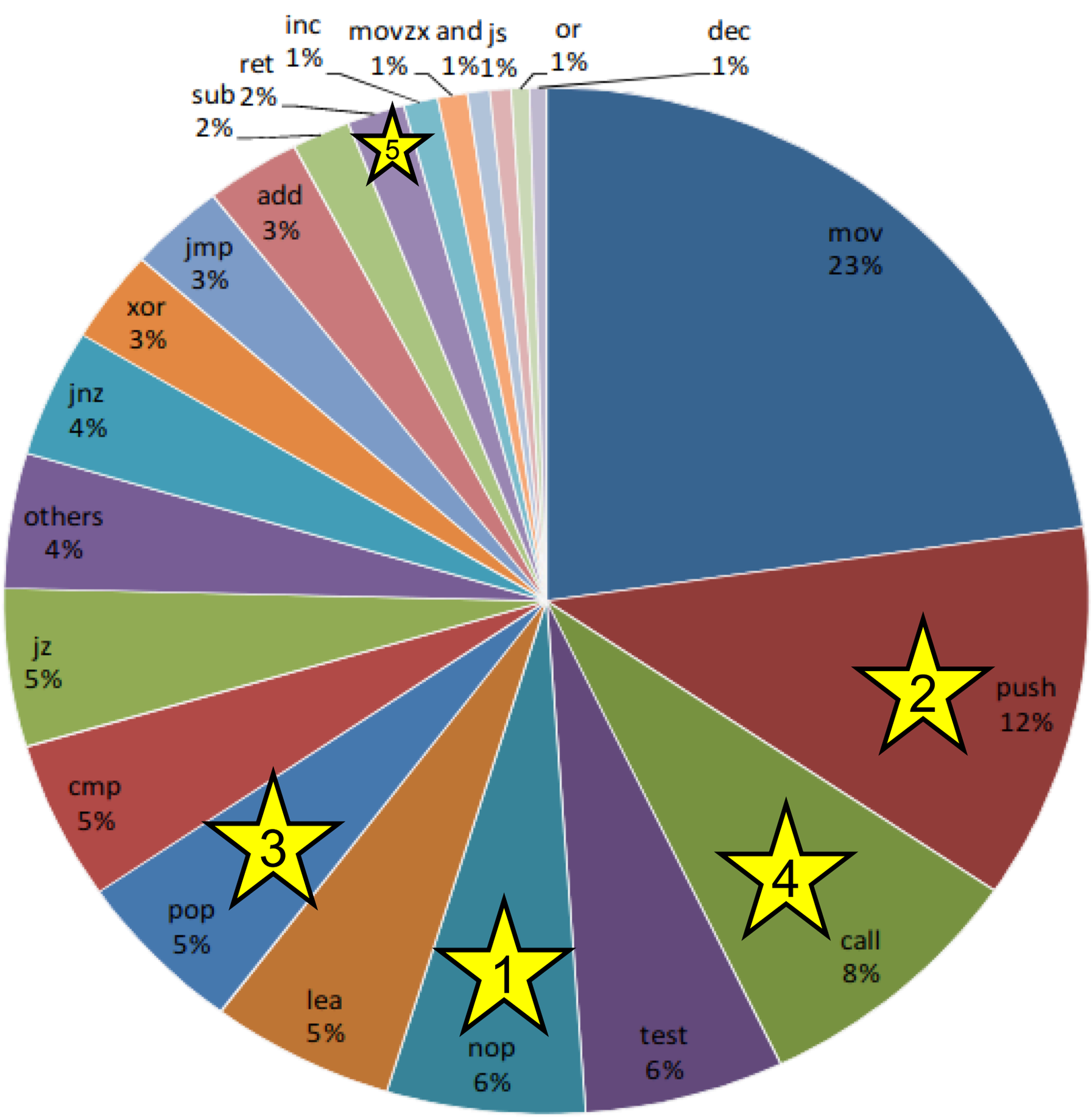


Fig. 2 Web browsers' instruction frequencies

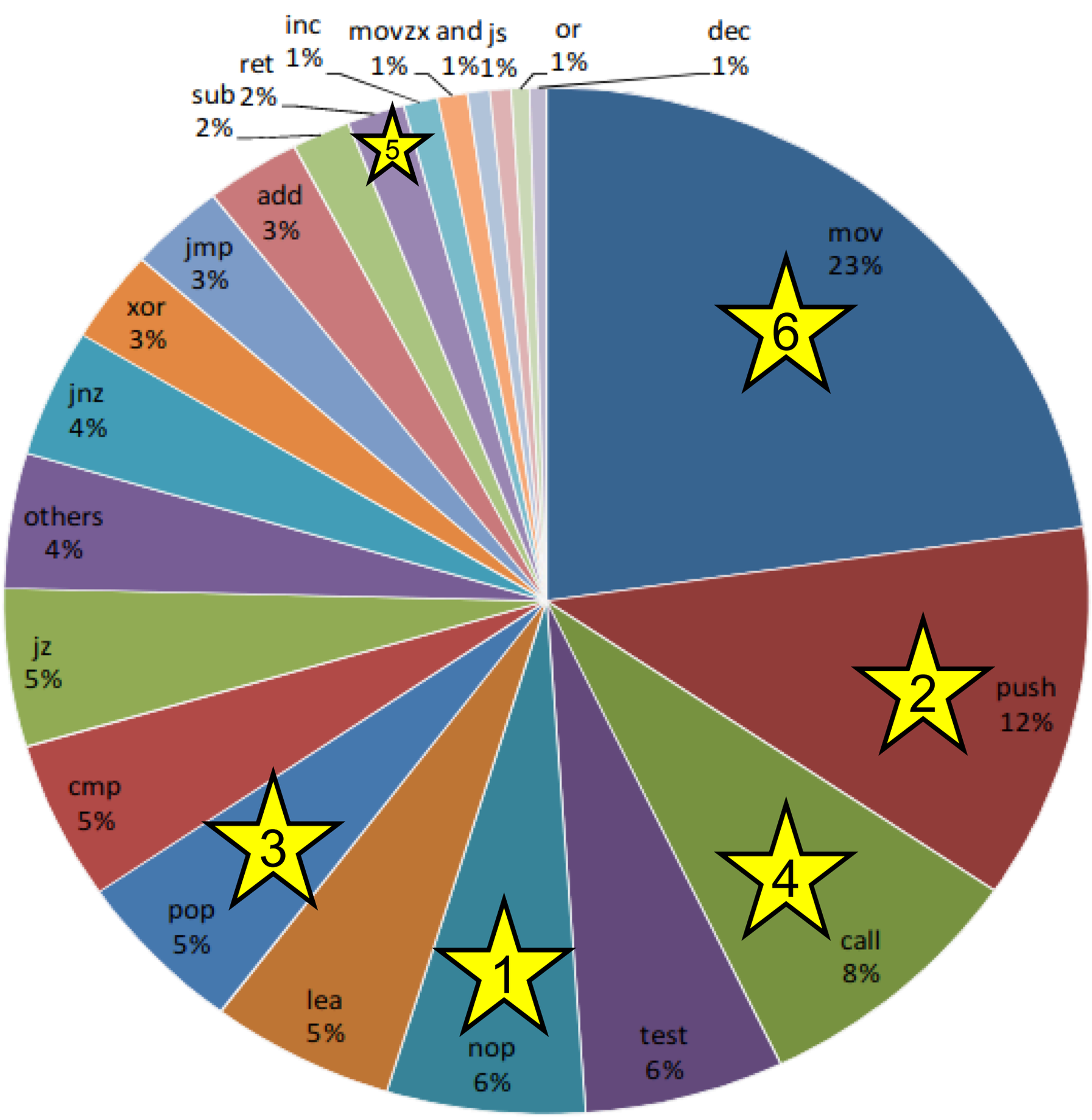


Fig. 2 Web browsers' instruction frequencies

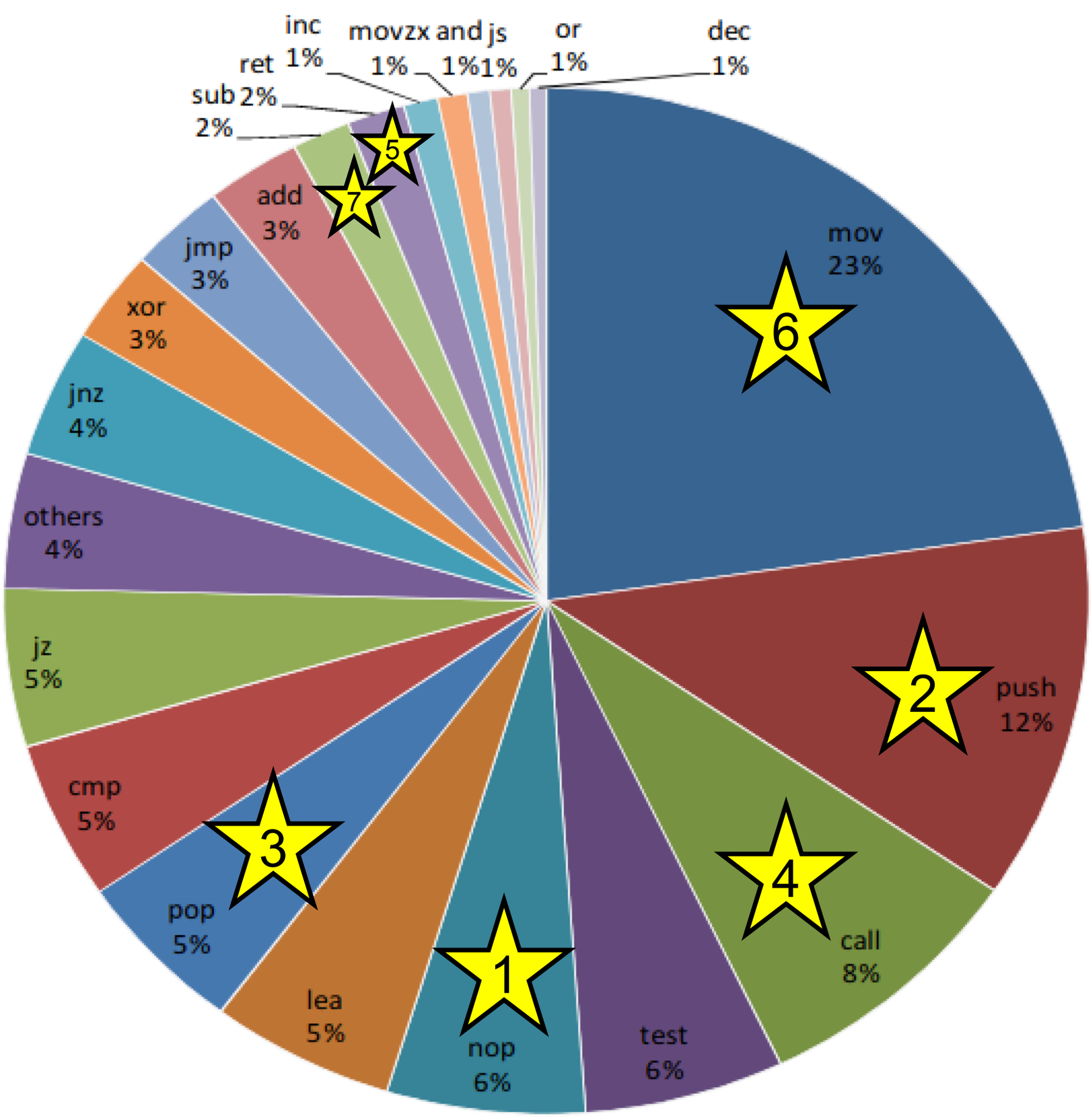


Fig. 2 Web browsers' instruction frequencies

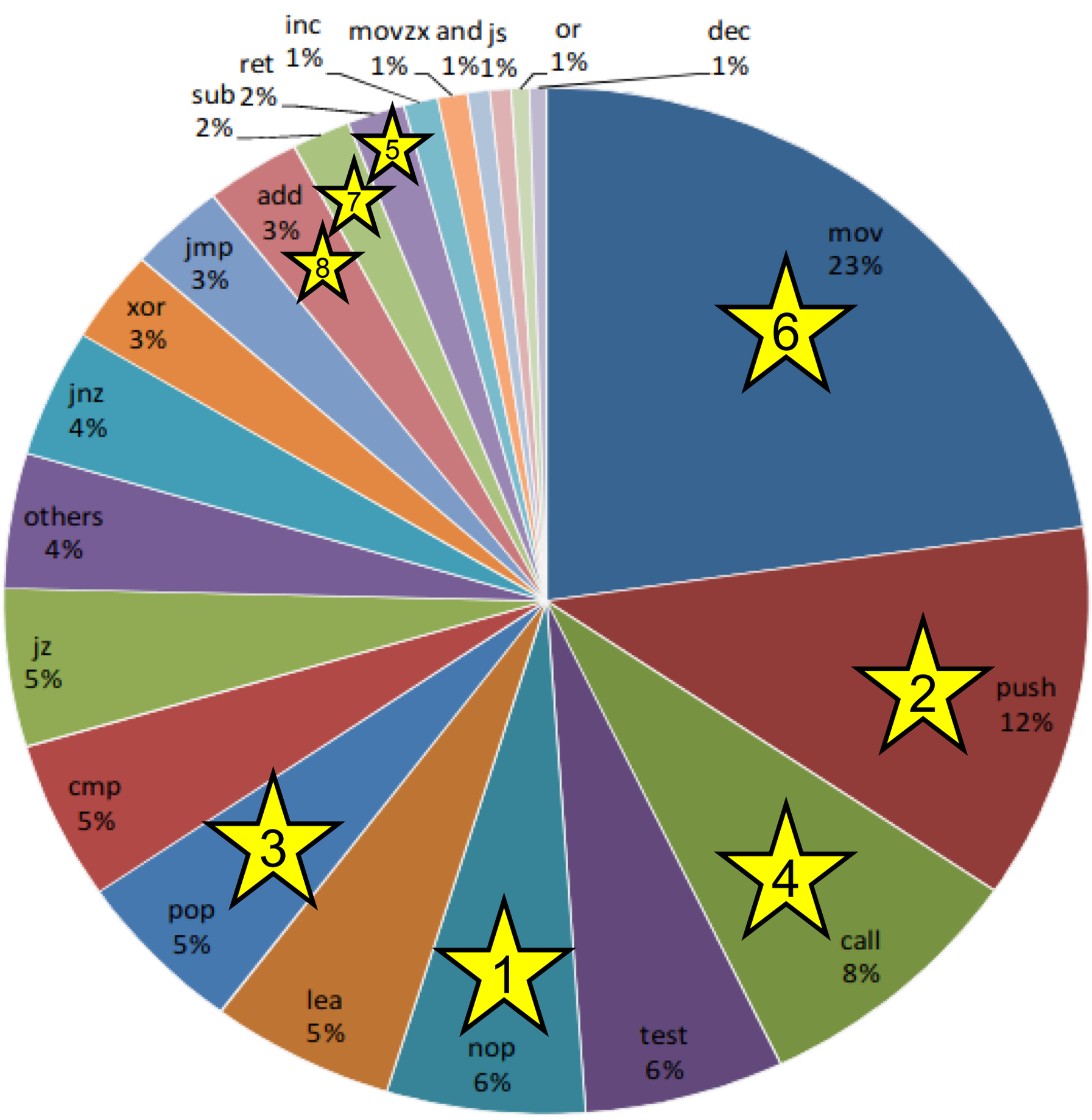


Fig. 2 Web browsers' instruction frequencies